

AIRCOORD

Delay-Aware State Reconstruction for MPC-Based Multi-Vehicle Coordination

Fatih Emin Uluşık

Technical research demo report

Core question. How much of the performance loss caused by delayed state reports can be recovered by reconstructing the present state from the command history before solving the same centralized MPC problem?

Abstract

I developed AIRCOORD as a compact simulation study of delayed information in multi-vehicle coordination. Four vehicles cross a shared conflict region while a centralized nonlinear model predictive controller (MPC) modifies a nominal goal-tracking command to maintain separation. I compare three information conditions: a current-state reference, direct use of stale reports, and command-history-based compensation that advances a delayed report to the present using the nominal dynamics. The plant is intentionally perturbed by unobserved acceleration noise so that compensation is useful but imperfect.

Across ten shared seeds, stale-state control remains acceptable at 0.5 s delay, begins to lose task completion at 1 s, and degrades sharply at 2 s: 7 of 10 episodes contain a separation violation and only 60% of vehicles arrive. Under the same 2 s condition, compensated control records no separation violations in the tested episodes and 100% vehicle arrival. The result is not a formal safety guarantee; it is a controlled baseline showing that explicitly treating information age can materially change closed-loop behavior. I use the model-based controller deliberately as a transparent baseline for later learning-based and uncertainty-aware extensions.

1 Problem formulation

A centralized coordinator receives vehicle states with an information delay. If it optimizes directly from a report that is already old, the commanded maneuver can be appropriate for the reported geometry but wrong for the current geometry. AIRCOORD isolates this effect without introducing a learned policy, perception stack, or communication protocol.

For vehicle i , I use the planar state and input

$$\mathbf{x}_i = [x_i, y_i, v_i, \psi_i]^\top, \quad \mathbf{u}_i = [a_i, \omega_i]^\top, \quad (1)$$

where a_i is longitudinal acceleration and ω_i is heading rate. With sample time Δt , the simulator first updates speed and heading and then advances position:

$$v_{i,k+1} = \text{clip}(v_{i,k} + (a_{i,k} + w_{i,k})\Delta t, 0, v_{\max}), \quad (2)$$

$$\psi_{i,k+1} = \text{wrap}(\psi_{i,k} + \omega_{i,k}\Delta t), \quad (3)$$

$$\mathbf{p}_{i,k+1} = \mathbf{p}_{i,k} + v_{i,k+1} \begin{bmatrix} \cos \psi_{i,k+1} \\ \sin \psi_{i,k+1} \end{bmatrix} \Delta t. \quad (4)$$

The disturbance $w_{i,k}$ is independent Gaussian acceleration noise applied only to the plant. The controller and the compensator do not observe its realized value.

The task is not only to avoid close approaches. A valid coordination method should also allow the vehicles to reach their goals. I therefore report both *separation* and *completion* metrics throughout the study.

2 Controller architecture

2.1 Nominal tracking

The nominal controller steers each vehicle toward its opposite goal while regulating speed toward 10 m/s. It contains no collision avoidance. This makes the role of the MPC correction visible rather than hiding avoidance behavior inside the reference policy.

2.2 Centralized MPC correction

At each control step, the MPC optimizes the active vehicles jointly. A 5 s horizon is represented by five piecewise-constant control blocks. Only the first command is executed, after which the optimization is repeated.

The decision objective is

$$J = J_{\text{track}} + 0.1J_{\text{smooth}} + 1000(s + s^2), \quad (5)$$

where J_{track} penalizes deviation from the nominal command, J_{smooth} penalizes block-to-block command variation, and $s \in [0, 1]$ is a global soft-constraint slack.

The planning distance is

$$D = d_{\min} + m = 20 \text{ m} + 3 \text{ m} = 23 \text{ m}, \quad (6)$$

and predicted closest pairwise distances must satisfy

$$\frac{d_{ij}^{\text{pred}}}{D} - 1 + s \geq 0. \quad (7)$$

I solve the nonlinear program with SLSQP. When an optimization is rejected, the controller applies maximum braking while retaining the nominal turn direction. This fallback is intentionally simple and is not claimed to be safe.

2.3 Delayed-state compensation

I compare three information conditions:

- **Current:** MPC receives the true current simulator state. This is a delay-free reference and is evaluated only once.
- **Stale:** MPC receives the historical state and active mask from the delayed report timestamp.
- **Compensated:** I start from the same delayed report and replay every command sent since that timestamp using the nominal model.

For a delayed report at time $k - L$, the compensator recursively applies the stored commands

$$\hat{\mathbf{x}}_{k-L} \rightarrow \hat{\mathbf{x}}_{k-L+1} \rightarrow \cdots \rightarrow \hat{\mathbf{x}}_k \quad (8)$$

with the same deterministic update model used by the controller. It does *not* access the current simulator state or the realized disturbances. Therefore, under exact disturbance-free dynamics it can reconstruct the present state, while unobserved disturbances produce a state-estimation error. This distinction is the mechanism being tested.

3 Experimental design

The four vehicles start near equally spaced points on a circle and travel toward opposite goals. I use the same seeds across methods so that initial-position jitter and plant disturbances are paired between conditions.

Table 1: Simulation and controller settings.

Parameter	Value
Vehicles / simulation horizon	4 / 60 s
Simulation step	0.1 s
Initial radius / coordinate jitter	150 m / ± 5 m
Cruise / maximum speed	10 m/s / 15 m/s
Acceleration command bound	$\pm 2 \text{ m/s}^2$
Heading-rate bound	$\pm 30 \text{ deg/s}$
Required separation / goal radius	20 m / 5 m
Plant acceleration disturbance	Gaussian, $\sigma = 0.3 \text{ m/s}^2$ per vehicle and step
MPC horizon / control blocks	5 s / 5
Additional planning margin	3 m
Optimizer	SLSQP, 80-iteration limit, warm initialization

A separation violation is recorded if any pair of vehicles that is active at the beginning of a step approaches within 20 m during that step. I compute closest approach along the linear segment swept during the discrete transition rather than only at sampled endpoints. Arrival rate is the fraction of vehicles that enter their goal disks within the 60 s simulation window.

4 Results

Table 2 reports the existing ten-seed comparison. The current-state condition is shown once because it does not depend on the configured report delay.

The main transition occurs between 1 s and 2 s. At 2 s, direct use of stale reports produces a separation violation in 70% of the tested episodes and only 24 of 40 vehicles reach their goals. Command replay changes that outcome substantially in the same paired scenarios: all 40 vehicles arrive and no episode crosses the 20 m threshold.

Table 2: Closed-loop results over ten shared seeds. “Violation” is episode-level; arrival rate is computed over 40 vehicles per delayed condition.

Condition	Episodes with violation	Arrival rate	Worst separation	Solver failures
Current reference	0/10	100.0%	22.896 m	1
Stale, 0.5 s	0/10	100.0%	21.399 m	10
Compensated, 0.5 s	0/10	100.0%	22.938 m	1
Stale, 1.0 s	0/10	97.5%	23.692 m	26
Compensated, 1.0 s	0/10	100.0%	22.865 m	0
Stale, 2.0 s	7/10	60.0%	3.980 m	34
Compensated, 2.0 s	0/10	100.0%	22.685 m	1

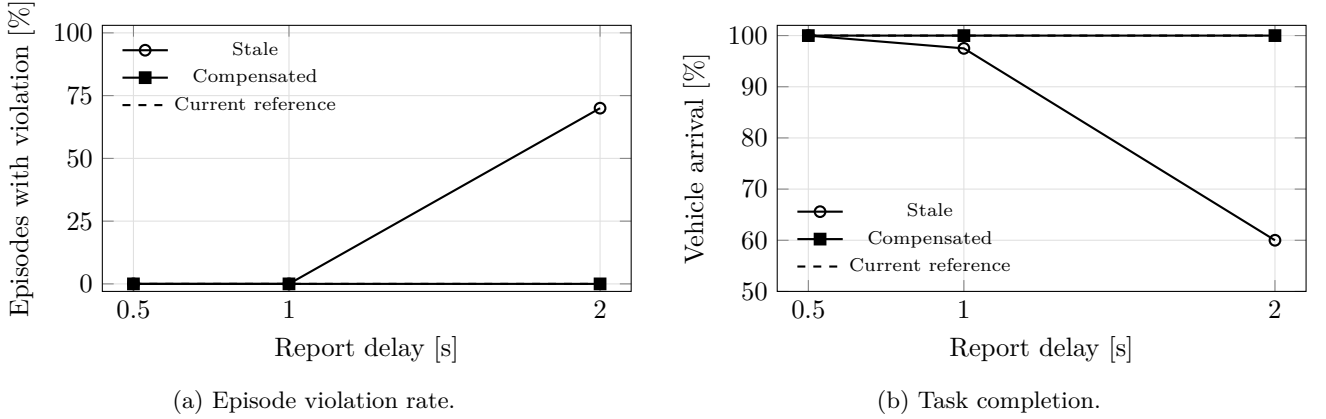


Figure 1: Observed sensitivity to report delay. The present sweep has only three delayed operating points, so it shows a transition but does not locate its onset precisely.

The 1 s stale result deserves separate interpretation. Its worst separation, 23.692 m, is larger than the current-state reference, yet one vehicle fails to arrive. Minimum separation is therefore not a monotonic measure of coordination quality: a slower or more circuitous trajectory can create more spacing while reducing task completion. The saved aggregate metrics do not identify the exact cause of that missed arrival, so I treat the 1 s result as an early coordination degradation rather than as an improvement.

Figure 2 shows one paired example at 2 s. For seed 7, stale-state MPC reaches a minimum separation of 6.823 m and only 75% arrival. With the same seed and delay, compensation keeps the closest approach at 22.795 m and all vehicles arrive. The paths also show that compensation is not merely adding conservative braking; it preserves a direct crossing pattern while changing the timing and geometry of the conflict-resolution maneuver.

5 What the experiment establishes

The strongest result is not that MPC is “safe,” and not that command replay solves delayed coordination in general. The result is narrower and more useful: *with the same controller, same nominal policy, same scenario seeds, and same unobserved plant disturbances, changing only the treatment of delayed information can move the system from severe failure to successful completion in the tested 2 s cases.*

This is why I intentionally keep the present version model-based. A learned policy would add a second source of approximation and make it harder to determine whether an observed change comes from the information interface or from policy learning. The current controller is therefore a baseline against which later learned components can be compared.

The experiment also exposes two weaknesses that should remain visible. First, the MPC uses soft separation constraints, so optimizer success does not imply a hard safety guarantee. Second, the simulator waits for the optimizer. Recorded single-decision times can exceed the 0.1 s simulation step, including a maximum of approximately 28.4 s in the 2 s stale condition. The current implementation should therefore be read as an algorithmic simulation study, not a real-time controller demonstration.

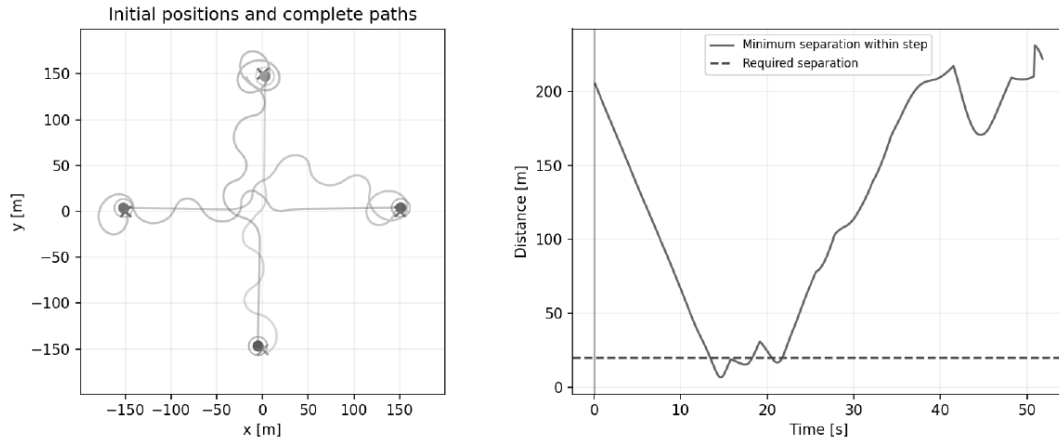
6 Next experiments

The next version should improve the evidence before making the controller more complicated.

1. **Resolve the delay transition.** Extend the sweep to at least $\{0.5, 1.0, 1.5, 2.0, 2.5, 3.0\}$ s and run 30–50 paired

Stale reports: 0.023 m minimum separation; 75% arrival.

AIRCOORD | Centralized MPC / stale / delay=2s | seed 7



Compensated reports: 22.795 m minimum separation; 100% arrival.

AIRCOORD | Centralized MPC / compensated / delay=2s | seed 7

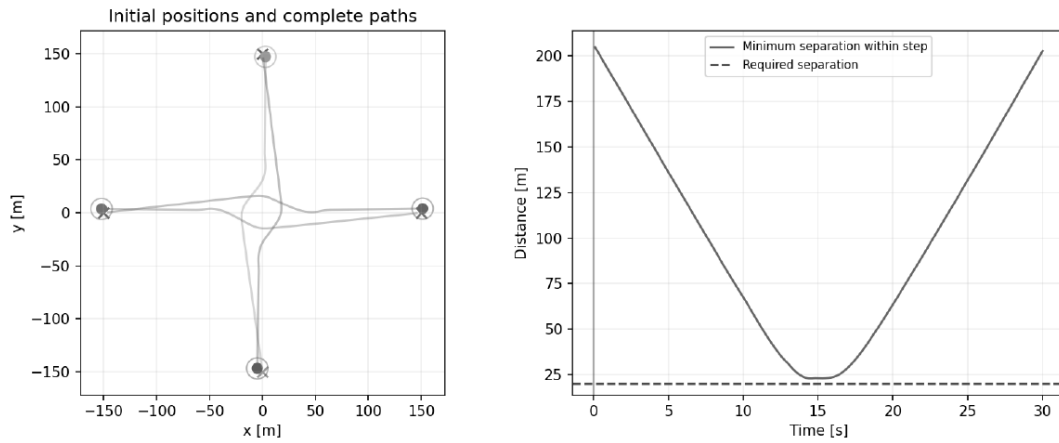


Figure 2: Representative seed-7 trajectories at 2s report delay. Top: stale reports. Bottom: command-history compensation. The dashed horizontal line in the right-hand plots marks the required 20 m separation.

seeds per condition. Report confidence intervals for episode violation and vehicle arrival rather than only point estimates.

2. **Measure the compensation mechanism directly.** Log $\|\hat{\mathbf{p}}_i - \mathbf{p}_i\|$ and heading/speed estimation errors over time, together with active-mask disagreements. This would connect closed-loop performance to the quantity the compensator is actually trying to reduce.
3. **Make the margin uncertainty-aware.** Replace the fixed 3 m margin with a margin or chance constraint driven by estimated reconstruction uncertainty. Chance-constrained air-traffic optimization under uncertain occupancy times provides a natural reference point for this direction [2].
4. **Introduce learning without losing the baseline.** A learned decentralized policy or residual model can generate the nominal action, while the current MPC remains a correction layer or teacher. Architecturally, this is related to action-filtering approaches in constrained multi-agent reinforcement learning, such as MAF-Net [1]. The comparison should be against the present deterministic baseline, not in place of it.
5. **Separate safety from numerical failure.** Replace the braking fallback with a separately tested backup strategy and integrate solver latency into the simulated timeline. A predictive safety-filter formulation is one relevant reference for structuring backup behavior and recursive feasibility [3].

These extensions keep the project focused: delayed information, uncertainty, and multi-agent coordination remain the core problem, while learning is added only after the baseline mechanism is measurable.

7 Reproducibility

I repeated the seed-7, 2 s, $\sigma_a = 0.3 \text{ m/s}^2$ cases for the three information conditions and recovered the stored trajectory-level outcomes: approximately 22.999 m minimum separation with 100% arrival for current state, 6.823 m with 75% arrival for stale state, and 22.795 m with 100% arrival for compensated state. The experiment uses fixed seeds and a disturbance stream indexed by simulation step and vehicle, so corresponding runs receive the same disturbance realization even when their closed-loop trajectories differ.

8 Conclusion

AIRCOORD demonstrates a specific failure mode in multi-vehicle MPC: stale state information can become the dominant source of error even when the controller itself is unchanged. Replaying the known command history is a simple model-based correction, but in this experiment it is enough to recover both separation and task completion at a delay where direct stale-state control breaks down. The next question is not whether to add more algorithmic complexity immediately, but how the reconstruction error grows with delay and disturbance, and how that uncertainty should enter the coordination constraint.

References

- [1] B. Pang, M. Zhang, X. Hu, D.-T. Pham, S. Alam, and G. Lulli, “Constrained Multi-Agent Reinforcement Learning with MAF-Net for Safe Trajectory Planning,” *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2026.
- [2] B. Pang, K. H. Low, and V. N. Duong, “Chance-constrained UAM traffic flow optimization with fast disruption recovery under uncertain waypoint occupancy time,” *Transportation Research Part C: Emerging Technologies*, vol. 161, 104547, 2024.
- [3] K. P. Wabersich and M. N. Zeilinger, “A predictive safety filter for learning-based control of constrained nonlinear dynamical systems,” *Automatica*, vol. 129, 109597, 2021.
- [4] C. E. Luis, M. Vukosavljev, and A. P. Schoellig, “Online trajectory generation with distributed model predictive control for multi-robot motion planning,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 604–611, 2020.